

Chapitre 7 : Compilation et architecture modulaire

I Introduction à la compilation

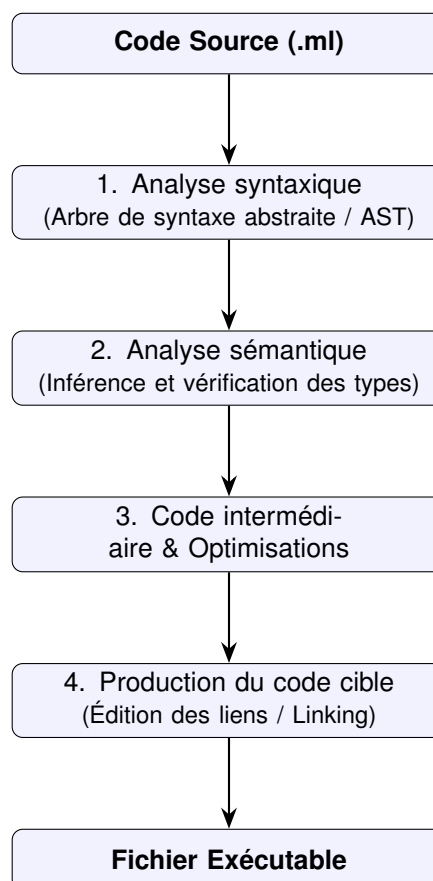
La **compilation** est la traduction d'un code source OCaml en un code exécutable cible (binaire machine ou code-octet).

Remarque : La compilation offre deux avantages par rapport à l'évaluation interactive (REPL) :

- **Autonomie** : Génération d'un fichier exécutable autonome sans dépendance à l'environnement OCaml complet.
- **Efficacité** : Application d'optimisations statiques et vitesse d'exécution accrue.

A Étapes de la compilation

Le compilateur OCaml applique un traitement séquentiel structuré en plusieurs phases :



B Les deux formats de cibles d'OCaml

1. **Le code-octet (*byte-code*)** : Compilateur `ocamlc`. Produit un code intermédiaire exécuté par la machine virtuelle OCaml. Fortement portable, temps de compilation minimal, mais exécution plus lente.
2. **Le code natif** : Compilateur `ocamlopt`. Produit un binaire machine spécifique à l'architecture cible (x86, ARM, etc.). Performances optimales, exécutable totalement indépendant.

II Interagir avec la ligne de commande : le module Sys

Un exécutable compilé est évalué séquentiellement de haut en bas. L'interaction avec l'environnement s'effectue via les arguments passés au processus.

Méthode : Le tableau des arguments Sys.argv

La valeur Sys.argv, de type string array, stocke les arguments de la ligne de commande :

- Array.length Sys.argv renvoie le nombre total d'arguments (nom du programme inclus).
- Sys.argv.(0) contient le nom ou le chemin de l'exécutable invoqué.
- Sys.argv.(i) (pour $i \geq 1$) contient le i -ème argument textuel.

💡 **Exemple** : Programme argsimple.ml d'affichage des arguments :

```
1 let () =
2   let nb_args = Array.length Sys.argv in
3   for i = 0 to nb_args - 1 do
4     Printf.printf "Argument numero %d : %s\n" i Sys.argv.(i)
5   done
```

Compilation et exécution dans le terminal :

```
1 $ ocamlc -o argsimple argsimple.ml
2 $ ./argsimple un deux trois
3 Argument numero 0 : ./argsimple
4 Argument numero 1 : un
5 Argument numero 2 : deux
6 Argument numero 3 : trois
```

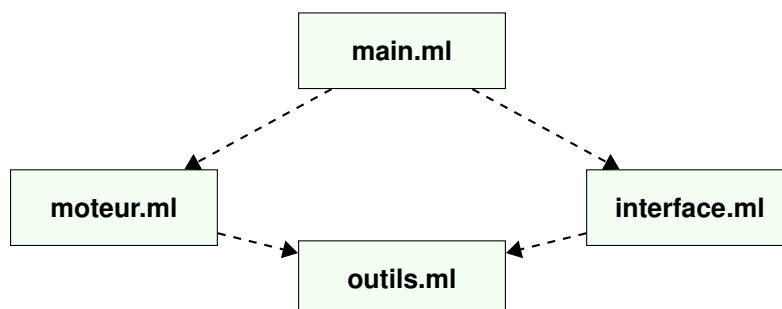
III La programmation modulaire

L'architecture d'un projet OCaml repose sur un découpage en unités logiques appelées **modules**.

A Architecture et graphe de dépendances

Un module regroupe des définitions de types, de valeurs, de fonctions et d'exceptions. Les dépendances inter-modules doivent former un **graphe orienté acyclique** (DAG). Les dépendances cycliques sont interdites à la compilation.

Définition : Le module A **dépend** du module B si A référence une entité exportée par l'interface de B .



B Les Interfaces en OCaml (.mli) : Rôles et Utilisation

Chaque module est scindé en deux composants : l'**implémentation** (.ml) et l'**interface** (.mli).

1 Encapsulation et masquage

L'interface agit comme un filtre d'accès. Toute entité définie dans le fichier `.ml` qui n'est pas explicitement déclarée dans le fichier `.mli` devient privée et inaccessible à l'extérieur du module.

2 Abstraction des types

Le fichier `.mli` permet de déclarer des **types abstraits**. En masquant l'implémentation concrète d'un type, on empêche l'extérieur de le manipuler directement ou d'effectuer du filtrage de motif dessus, garantissant l'invariance des données.

 **Exemple** : Spécification abstraite (`tour.mli`) et implémentation concrète (`tour.ml`) :

Fichier d'interface : `tour.mli`

```
1 type tower (* Type abstrait *)
2
3 exception Operation_illegal
4
5 val empty : tower
6 val push : int -> tower -> tower
7 val pop : tower -> tower
8 val top : tower -> int
```

Fichier d'implémentation : `tour.ml`

```
1 type tower = int list (* Structure de donnees concrete *)
2
3 exception Operation_illegal
4
5 let empty = []
6
7 let push x t =
8   match t with
9   | [] -> [x]
10  | h :: _ -> if x < h then x :: t else raise Operation_illegal
11
12 let pop t =
13   match t with
14   | [] -> raise Operation_illegal
15   | _ :: r -> r
16
17 let top t =
18   match t with
19   | [] -> raise Operation_illegal
20   | h :: _ -> h
```

 **Remarque** : L'abstraction de type assure le découplage : une modification de la structure interne dans le fichier `.ml` (par exemple remplacer une liste par un tableau) n'impacte aucunement les modules clients tant que le fichier `.mli` reste inchangé.

3 Ouverture de modules (`open`)

Par défaut, l'accès aux éléments d'un module s'effectue par nommage qualifié (ex: `Module.valeur`). La première lettre du nom du module est obligatoirement une majuscule.

Méthode : L'instruction `open`

L'expression `open NomDuModule` importe l'ensemble du scope public du module dans l'environnement courant, dispensant de l'usage du préfixe qualificatif.

💡 **Exemple** : Syntaxe d'importation dans `main.ml` :

```

1 (* Approche 1 : Accès qualifié (Recommande pour la maintenance) *)
2 let pile1 = Tour.push 5 Tour.empty
3
4 (* Approche 2 : Ouverture globale du scope *)
5 open Tour
6 let pile2 = push 5 empty

```

✗ **Attention** ✗ L'ouverture globale via `open` peut induire des conflits de noms (masquage de fonctions homonymes). Il est recommandé d'utiliser des ouvertures locales restreintes : `let open Tour in push 5 empty` ou la syntaxe compacte `Tour.(push 5 empty)`.

C Le mécanisme de compilation séparée

La compilation séparée permet de compiler un module indépendamment de l'implémentation des modules dont il dépend : seule la connaissance de leurs interfaces est requise.

Méthode : Processus et extensions de fichiers

Le compilateur génère des fichiers intermédiaires spécifiques :

- `ocamlc B.mli` → compile l'interface et génère le fichier binaire d'interface `B.cmi`.
- `ocamlc -c B.ml` → compile le corps du module (option `-c`, sans édition de liens) et génère le fichier objet `B.cmo` (`B.cmx` en code natif).

L'édition des liens rassemble les objets compilés pour produire l'exécutable :

`ocamlc -o mon_programme B.cmo A.cmo.`

✗ **Attention** ✗ L'ordre des fichiers objets lors du linking est strict : un module *B* requis par un module *A* doit impérativement précéder *A* dans la commande de liaison.

IV Automatisation du build avec `make`

L'outil `make` automatise la construction en résolvant le graphe de dépendances et en limitant la recompilation aux seuls fichiers modifiés ou impactés par une modification amont.

💡 **Exemple** : Makefile générique pour le projet `tour` :

```

1 # Règle principale : Edition des liens
2 main: tour.cmo main.cmo
3     ocamlc -o main tour.cmo main.cmo
4
5 # Compilation de l'interface
6 tour.cmi: tour.mli
7     ocamlc -c tour.mli
8
9 # Compilation de l'implémentation (depend de sa propre interface)
10 tour.cmo: tour.cmi tour.ml
11     ocamlc -c tour.ml
12
13 # Compilation du programme principal (depend de l'interface de Tour)
14 main.cmo: tour.cmi main.ml
15     ocamlc -c main.ml
16
17 # Nettoyage des cibles intermédiaires

```

```
18 .PHONY: clean
19 clean:
20     rm -f main *.cmi *.cmo
```

Contents

I	Introduction à la compilation	1
A	Étapes de la compilation	1
B	Les deux formats de cibles d'OCaml	1
II	Interagir avec la ligne de commande : le module <code>Sys</code>	2
III	La programmation modulaire	2
A	Architecture et graphe de dépendances	2
B	Les Interfaces en OCaml (<code>.mli</code>) : Rôles et Utilisation	2
1	Encapsulation et masquage	3
2	Abstraction des types	3
3	Ouverture de modules (<code>open</code>)	3
C	Le mécanisme de compilation séparée	4
IV	Automatisation du build avec <code>make</code>	4